

■ POSIX Threads: Condition Variables

Beginner Exam Cheat Sheet · Mutexes · Predicates · pthread_cond · Producer-Consumer

1 · Core Concepts

What is a Thread?

A **thread** is a lightweight unit of execution that shares the same memory space as other threads in the same process. Multiple threads run (seemingly) simultaneously and can read/write the same variables — which is powerful but dangerous without proper synchronization.

What is a Mutex?

A **mutex (mutual exclusion lock)** ensures that only ONE thread at a time can enter a **critical section** — a piece of code that accesses shared data. Any other thread trying to lock the same mutex will *block* (pause) until the lock is released.

Mutex key rules:

- Lock *before* accessing shared data
- Unlock *immediately after* you're done
- Never access shared data without the lock
- A thread that already holds the lock will deadlock if it tries to lock again

Basic mutex usage:

```
pthread_mutex_t m;  
pthread_mutex_init(&m;, NULL);  
  
pthread_mutex_lock(&m;);    // LOCK  
/* critical section */  
pthread_mutex_unlock(&m;);  // UNLOCK  
  
pthread_mutex_destroy(&m;);
```

The Producer-Consumer Problem

A classic multithreading scenario: one or more **producer** threads create data and put it into a shared buffer; one or more **consumer** threads take data out of the buffer and process it. The buffer has a maximum size (bounded buffer).

- **Constraint 1 – Mutual Exclusion:** Only one thread may touch the buffer at a time.
- **Constraint 2 – Synchronization:** Consumer must wait if buffer is empty; producer must wait if buffer is full.

■ Why can't we just use a mutex alone?

A mutex only prevents simultaneous access. It does NOT let a thread efficiently wait for a condition (e.g., 'wait until data is ready'). Without condition variables, a thread must spin-poll (busy-wait), wasting CPU. Condition variables let the OS put a waiting thread to sleep and wake it only when relevant.

2 · Naive Approach — Busy-Wait (Don't Use This!)

Before condition variables, you might try a spin-poll loop. This works but wastes CPU by repeatedly locking/unlocking the mutex even when nothing has changed.

```
// BAD: busy-wait consumer  
int fetched = 0;  
do {  
    pthread_mutex_lock(&buffer._lock);
```

```

if (nElements > 0) {                // Is there data?
    data = fetch_from_buffer();
    fetched = 1;
}
pthread_mutex_unlock(&buffer;_lock);
} while (fetched == 0);              // Keep trying if not fetched
// Problem: burns CPU looping even when buffer is always empty

```

■ ■ **Problem** If the buffer stays empty for a long time, this loop runs millions of times, burning CPU cycles and slowing down the producer too.

3 · Condition Variables — The Right Tool

A **condition variable (CV)** is a synchronization object that lets a thread *sleep efficiently* while waiting for a condition to become true, and lets other threads *wake it up* when that condition changes.

Key idea:

- A CV is always paired with a mutex — you **must** hold the mutex when you call wait/signal.
- When a thread calls **wait**, it atomically (1) releases the mutex and (2) goes to sleep. When woken, it re-acquires the mutex before returning.
- Another thread calls **signal** or **broadcast** to wake sleeping threads after changing shared state.

What is a Predicate?

A **predicate** is a boolean condition (true/false) that describes the state of shared data that a thread needs to be true before it can proceed.

Examples of predicates:

- "Buffer is not empty" → consumer can fetch
- "Buffer is not full" → producer can insert
- "task_available == 1" → consumer has work
- "result_ready == true" → display thread can render

Why use a while loop, not if?

Always check the predicate in a while loop after waking up because of **spurious wake-ups** — the OS can wake a thread even when no one called signal. Re-checking prevents acting on stale state.

■ **Golden Rule** ALWAYS check your predicate in a WHILE loop, never an IF. Even after being woken up by a signal, verify the predicate is actually true before proceeding.

4 · POSIX Condition Variable API

Function	What it does	When to call it
pthread_cond_init (&cond;, NULL)	Initializes the condition variable. NULL for default attributes.	Once, before any thread uses the CV.
pthread_cond_destroy (&cond;)	Frees resources associated with the CV.	Once, after all threads are done.
pthread_cond_wait (&cond;, &mutex;)	Atomically releases mutex and sleeps. Re-locks mutex when woken.	Inside a while loop, after locking the mutex.
pthread_cond_signal (&cond;)	Wakes ONE waiting thread (if any). Does nothing if no thread is waiting.	After changing shared state, while holding the mutex.

pthread_cond_broadcast (&cond;)	Wakes ALL waiting threads. Only one can actually grab the mutex.	When multiple waiters might now be able to proceed.
------------------------------------	--	---

5 · The Universal Mutex + Condition Variable Pattern

Every correct use of condition variables follows this exact template. Memorize it!

```
pthread_mutex_t mutex;
pthread_cond_t cond;

/* ■■ WAITER (thread that needs a condition to be true) ■■ */
pthread_mutex_lock(&mutex);           // 1. Lock the mutex first
while (!predicate) {                  // 2. Check condition in a WHILE loop
    pthread_cond_wait(&cond, &mutex); // 3. Sleep: releases mutex, re-locks on wake
}                                     // 4. Re-check predicate (spurious wake-up safety)
/* access shared resources here */    // 5. Mutex is held; safe to proceed
pthread_mutex_unlock(&mutex);         // 6. Release mutex when done

/* ■■ SIGNALER (thread that changes the condition) ■■ */
pthread_mutex_lock(&mutex);           // 1. Lock the mutex
/* change shared state / update predicate */
pthread_cond_signal(&cond);          // 2. Wake one waiting thread
// OR pthread_cond_broadcast(&cond); // Wake ALL waiting threads
pthread_mutex_unlock(&mutex);         // 3. Release mutex
```

Step-by-step breakdown:

Waiter side:

- **Step 1:** Lock the mutex — required before calling wait.
- **Step 2:** Check the predicate. If already true, skip waiting.
- **Step 3:** Call wait — atomically releases mutex and sleeps.
- **Step 4:** On wake-up, mutex is re-locked. Re-check the predicate.
- **Step 5:** Predicate is true → safely access shared data.
- **Step 6:** Unlock mutex so others can proceed.

Signaler side:

- **Step 1:** Lock the mutex to safely modify shared state.
- **Step 2:** Update shared data (e.g., add item to buffer, set flag).
- **Step 3:** Call signal/broadcast to wake waiting threads.
- **Step 4:** Unlock the mutex.



Tip

Signal AFTER updating state, so the woken thread immediately

6 · Complete Producer-Consumer Example (Annotated)

Setup / Initialization (in main)

```
pthread_cond_t cond_empty;           // CV: 'buffer is empty' – producer waits on this
pthread_cond_t cond_full;           // CV: 'buffer is full' – consumer waits on this
pthread_mutex_t lock;               // Single mutex protecting everything
int task_available = 0;             // Predicate: 0 = empty, 1 = has data

int main() {
    pthread_cond_init(&cond_empty, NULL);
    pthread_cond_init(&cond_full, NULL);
    pthread_mutex_init(&lock, NULL);
    task_available = 0;
    /* create producer & consumer threads, then join them */
}
```

Producer Thread

```

void *producer(void *arg) {
    while (!done()) {
        create_task();           // Do this OUTSIDE the lock (time-consuming)

        pthread_mutex_lock(&lock);
        while (task_available == 1) // Buffer full: wait until consumer empties it
            pthread_cond_wait(&cond;_empty, &lock);

        insert_into_queue();      // Safe: we hold the lock
        task_available = 1;       // Update the predicate
        pthread_cond_signal(&cond;_full); // Tell consumer data is ready
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

```

Consumer Thread

```

void *consumer(void *arg) {
    while (!done()) {
        pthread_mutex_lock(&lock);
        while (task_available == 0) // Buffer empty: wait until producer fills it
            pthread_cond_wait(&cond;_full, &lock);

        my_task = extract_from_queue(); // Safe: we hold the lock
        task_available = 0;             // Update the predicate
        pthread_cond_signal(&cond;_empty); // Tell producer there's room
        pthread_mutex_unlock(&lock);

        process_task(my_task);          // Do this OUTSIDE the lock (time-consuming)
    }
    return NULL;
}

```

■ Design Rule Do time-consuming work (create_task, process_task) OUTSIDE the critical section. Keep the locked region as short as possible so other threads aren't blocked unnecessarily.

7 · Two Key Scenarios (Walk-Through)

Scenario 1 — Eager Producer (buffer full, producer must wait)

Step	Thread	Action	Predicate P
1	T1 (producer)	Has data to add. Locks mutex → enters CS.	P = True (buffer full)
2	T1 (producer)	Checks predicate P (buffer full?): YES → calls wait(C, mutex).	P = True
3	T1 (producer)	wait() atomically RELEASES mutex. T1 sleeps.	P = True
4	T2 (consumer)	Sees mutex is free → locks it → enters CS.	P = True
5	T2 (consumer)	Fetches data from buffer. Updates P = False. Calls signal(C).	P = False
6	T2 (consumer)	Unlocks mutex. Starts processing data.	P = False
7	T1 (producer)	Woken by signal. Re-locks mutex. Re-checks P (False) → proceeds.	P = False
8	T1 (producer)	Inserts new data. Unlocks mutex.	P = True

Scenario 2 — Eager Consumer (buffer empty, consumer must wait)

Step	Thread	Action	Predicate P
------	--------	--------	-------------

1	T2 (consumer)	Tries to fetch data. Locks mutex.	P = False (buffer empty)
2	T2 (consumer)	Checks predicate P (data ready?): NO → calls wait(C, mutex).	P = False
3	T2 (consumer)	wait() atomically RELEASES mutex. T2 sleeps.	P = False
4	T1 (producer)	Sees mutex is free → locks it → enters CS.	P = False
5	T1 (producer)	Inserts data. Sets P = True. Calls signal(C).	P = True
6	T1 (producer)	Unlocks mutex.	P = True
7	T2 (consumer)	Woken by signal. Re-locks mutex. Re-checks P (True) → proceeds.	P = True
8	T2 (consumer)	Fetches data. Sets P = False. Signals producer. Unlocks mutex.	P = False

8 · signal() vs. broadcast()

pthread_cond_signal()

- Wakes exactly **one** thread waiting on the CV.
- Which thread is woken is up to the OS scheduler.
- Use when only ONE waiting thread can usefully proceed (e.g., exactly one item in buffer).
- More efficient — no unnecessary wake-ups.

Example: One slot, one consumer waiting → signal is perfect.

pthread_cond_broadcast()

- Wakes **all** threads waiting on the CV.
- All woken threads compete for the mutex; only one wins.
- Losers re-check predicate, find it still false, go back to sleep.
- Use when multiple waiters might be able to proceed, or when unsure.

Caveat: A thread that calls wait AFTER a broadcast is NOT woken by that broadcast — it misses it.

9 · Common Mistakes & How to Avoid Them

Mistake	Why It's Wrong	Fix
Using if instead of while for predicate checks	Spurious wake-ups can occur. The OS may wake your thread even when the predicate is still false.	Always use while (predicate) { if (predicate) { ... } }
Calling wait without holding the mutex	Undefined behavior. The mutex/CV pair must be consistently locked.	Always lock the mutex before pthread_cond_wait() .
Forgetting to signal/broadcast after changing state	Waiting threads will sleep forever even though the condition is true.	Every time you change the predicate value, call signal or broadcast.
Signaling without holding the mutex	Race condition: a thread might be about to wait right as you signal.	Always hold the single mutex calling signal or broadcast.
Doing long work inside the critical section	Other threads are blocked waiting for the mutex the entire time.	Move non-critical work (I/O, computation) outside the lock.
Using the wrong condition variable	With two CVs (empty/full), signaling the wrong one wakes the wrong thread.	Use the CVs clearly . Producer waits on 'empty'; consumer waits on 'full'.
Forgetting to init/destroy mutex and CV	Using uninitialized synchronization primitives is undefined.	Call pthread_mutex_init and pthread_cond_init before use; destroy them when done.

10 · Quick-Reference Cheat Card

Declarations you always need:

```
#include
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // or use pthread_mutex_init
pthread_cond_t cond = PTHREAD_COND_INITIALIZER; // or use pthread_cond_init
int predicate = 0; // shared state variable
```

Waiter skeleton (copy-paste and adapt):

```
pthread_mutex_lock(&mutex);
while (!predicate) {                // WHILE — not if!
    pthread_cond_wait(&cond, &mutex);
}
/* do work with shared data */
pthread_mutex_unlock(&mutex);
```

Signaler skeleton (copy-paste and adapt):

```
pthread_mutex_lock(&mutex);
predicate = 1;                    // change shared state
pthread_cond_signal(&cond);       // wake one waiter
pthread_mutex_unlock(&mutex);
```

Exam Question Checklist:

- Is the predicate checked in a **while** loop? ✓
- Is the mutex locked before calling **wait**? ✓
- Is the mutex released after the critical section? ✓
- Is signal called after changing the predicate? ✓
- Is time-consuming work done **outside** the lock? ✓
- Are the correct CVs used (e.g., empty vs. full)? ✓
- Are init/destroy calls present? ✓
- Is broadcast used when **multiple** waiters could proceed? ✓

11 · Full pthread Mutex API (for reference)

Function	Description
pthread_mutex_init(&m, NULL)	Initialize mutex. NULL = default attributes. Returns 0 on success.
pthread_mutex_destroy(&m;)	Free mutex resources. Do not destroy a locked mutex.
pthread_mutex_lock(&m;)	Block until the mutex is available, then lock it.
pthread_mutex_trylock(&m;)	Try to lock. Returns EBUSY immediately if already locked (non-blocking).
pthread_mutex_unlock(&m;)	Release the mutex. Must be called by the thread that holds the lock.

12 · Mental Model Summary

Think of a mutex + condition variable as a **guarded waiting room**:

- The **mutex** is the door lock. Only one person (thread) can be inside at a time.
- The **condition variable** is a couch inside the room where you can nap while waiting.
- When you nap (wait), you automatically unlock the door so others can enter.
- When someone changes the situation (signals), they knock to wake you up.
- When you wake up, you re-lock the door and check if things are really ready.

Summary

Mutexes enforce exclusion. Condition variables add efficient waiting. Together they solve problems like producer-consumer where threads must wait for specific states without burning CPU. Always: LOCK → WHILE(!pred) WAIT → DO WORK → UNLOCK. Signal only after changing shared state. Use broadcast when multiple waiters could proceed.

Generated from lecture slides: Threads — POSIX Condition Variables · Good luck on your exam!